

Monte Carlo simulation CS 170 Lab 5

Learning objectives: By the end of the session students will be able to:

- Distinguish between a for loop and a while loop: use a for loop when the number of iterations is known in advance; use a while loop when the stopping point is determined by a condition
- Write a for loop with correct initialization, condition, and update expressions to repeat a fixed number of iterations
- Write a while loop that continues executing until a boolean condition becomes false, and identify what makes a loop terminate (or run forever)
- Use a for loop body to accumulate a running count across n iterations — the pattern used in the Monte Carlo simulation
- Use a while loop with two converging index variables (left and right) to traverse a string from both ends simultaneously — the pattern used in palindrome checking
- Trace a loop's execution step by step, showing the value of the control variable at each iteration, to verify correctness and termination

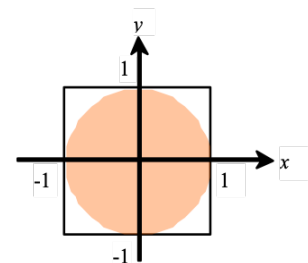
For loop vs while loop - when to use each

for loop	while loop
Use when the number of iterations is known before the loop starts.	Use when the stopping condition is checked and the count is not known in advance.
<pre>for (int i = 0; i < n; i++) { // body runs exactly n times }</pre>	<pre>while (condition) { // body runs until condition is false }</pre>
Q1 — Run exactly n trials in MyPI.java	Q2 — Advance left & right until they meet in Palindrome.java

Q1. The Monte Carlo simulation refers to a technique that uses random numbers and probability to solve problems. This method has a wide range of applications in computational mathematics, physics, chemistry, and finance. This section gives an example of using the Monte Carlo simulation for estimation.

Example picture shown on the right.

We can randomly scatter points inside a unit square (side length 1, like above picture). Simply count how many land inside the quarter-circle of radius 1 centered at the origin. Then we can estimate the PI based on the



ratio of hits to total points approximates the ratio of areas, like the given form here $PI \approx 4 * \text{numberOfHits} / \text{numberOfTrials}$, since $\text{circleArea} / \text{squareArea} = PI/4$.

Write a program `MyPI.java` to generate the PI as accurate as possible. Note you can randomly generate n points in square and to see how many points are in circle as well. The larger n is, the more accurate it should be. Your grade won't be judged based on the value of number n , but you can try this program with several different values for n .

Sample output

Input/Call	Expected output	Note
$n = 100$	$PI \approx 3.04$ (rough)	few trials — high variance
$n = 10,000$	$PI \approx 3.138$	much closer
$n = 1,000,000$	$PI \approx 3.14159$	very accurate

Tasks (complete in order):

(10 pts) Identify the geometric condition: for a random point (x, y) in the unit square, write the boolean expression that tells you the point is inside the quarter-circle of radius 1.

(20 pts) Implement the for loop counting body: declare an integer n (your chosen number of trials) and a counter `hits`. Write a forloop that runs exactly n times — use n as the loop bound. Inside the body, generate a random x and y and increment `hits` when the circle condition holds.

(10 pts) Compute and print the estimate: apply the formula $\pi \approx 4.0 * \text{hits} / n$. Print your estimate and `Math.PI` side by side.

(10 pts) Run the program with at least three different values of n (e.g. 100, 10 000, 1 000 000). Record the output in comments at the bottom of your file and describe the trend you observe.